In practice, `reload` is rarely used outside the context of unit tests.

Updating Existing Rows

After such a long discussion of finder methods, you'll be pleased to know that there's not much to say about updating records with Active Record.

If you have an Active Record object (perhaps representing a row from our `orders` table), you can write it to the database by calling its `save` method. If this object had previously been read from the database, this save will update the existing row; otherwise, the save will insert a new row.

If an existing row is updated, Active Record will use its primary key column to match it with the in-memory object. The attributes contained in the Active Record object determine the columns that will be updated—a column will be updated in the database only if its value has been changed. In the following example, all the values in the row for order 123 can be updated in the database table:

```
order = Order.find(123)
order.name = "Fred"
order.save
```

However, in the following example, the Active Record object contains just the attributes `id`, `name`, and `paytype`—only these columns can be updated when the object is saved. (Note that you have to include the `id` column if you intend to save a row fetched using `find_by_sql`.)

```
orders = Order.find_by_sql("select id, name, pay_type from orders where id=123")
first = orders[0]
first.name = "Wilma"
first.save
```

In addition to the `save` method, Active Record lets us change the values of attributes and save a model object in a single call to `update`:

```
order = Order.find(321)
order.update(name: "Barney", email: "barney@bedrock.com")
```